

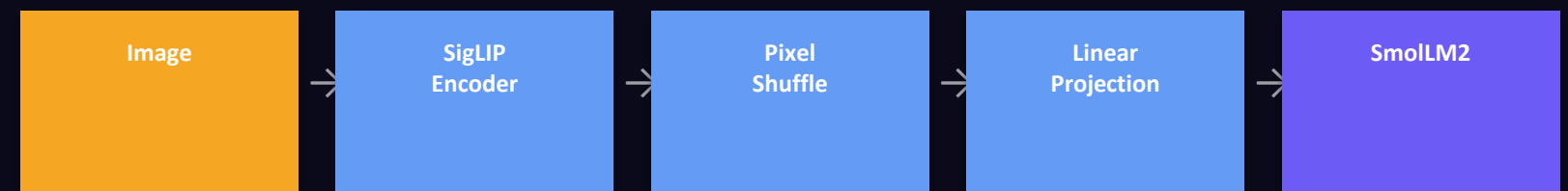
# SmolVLM

## Redefining Small and Efficient Multimodal Models

Hugging Face & Stanford University

*arXiv:2504.05299*

For ML Researchers | On-Device AI Engineers | Graduate Students



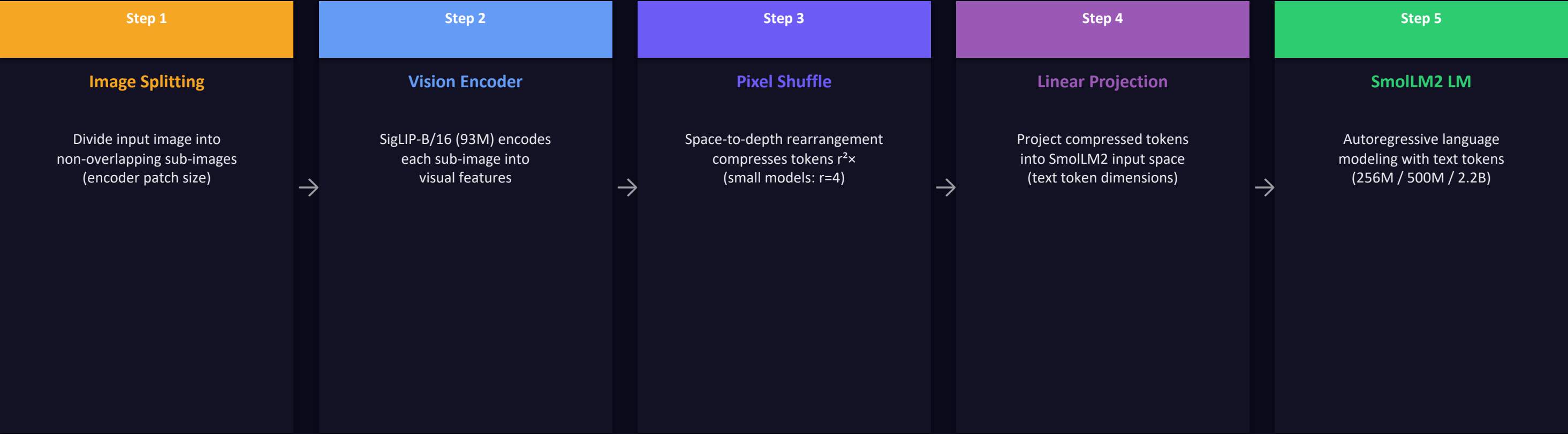
Architecture Pipeline

256M · 500M · 2.2B parameters | <1GB GPU RAM | Fully Open-Source

# Large VLMs Are Powerful but Impractical for Edge Deployment

| Model        | Parameters  | RAM (bs=1) | Efficiency      | <div><div>The Edge Deployment Problem</div><div><ul style="list-style-type: none"><li>• SOTA VLMs demand datacenter GPUs</li><li>• 27+ GB RAM excludes mobile/edge</li><li>• 80B+ params need quantization</li><li>• Cloud inference → latency + privacy issues</li><li>• SmolVLM: same task, pocket-sized</li></ul><div><div>SmolVLM-256M runs on smartphones with &lt;1GB RAM and surpasses the 300× larger Ldefics-80B</div></div></div></div> |
|--------------|-------------|------------|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Idefics-80B  | 80B params  | 27.7 GB    | <div></div>     |                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| Molmo-A1B-7B | 7B params   | 27.7 GB    | <div></div>     |                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| InternVL2-2B | 2B params   | 4.9 GB     | <div></div>     |                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| SmolVLM-2.2B | 2.2B params | 4.9 GB     | <div></div>     |                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| SmolVLM-500M | 500M params | 1.2 GB     | <div></div>     |                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| SmolVLM-256M | 256M params | 0.8 GB     | <div></div> 30% |                                                                                                                                                                                                                                                                                                                                                                                                                                                   |

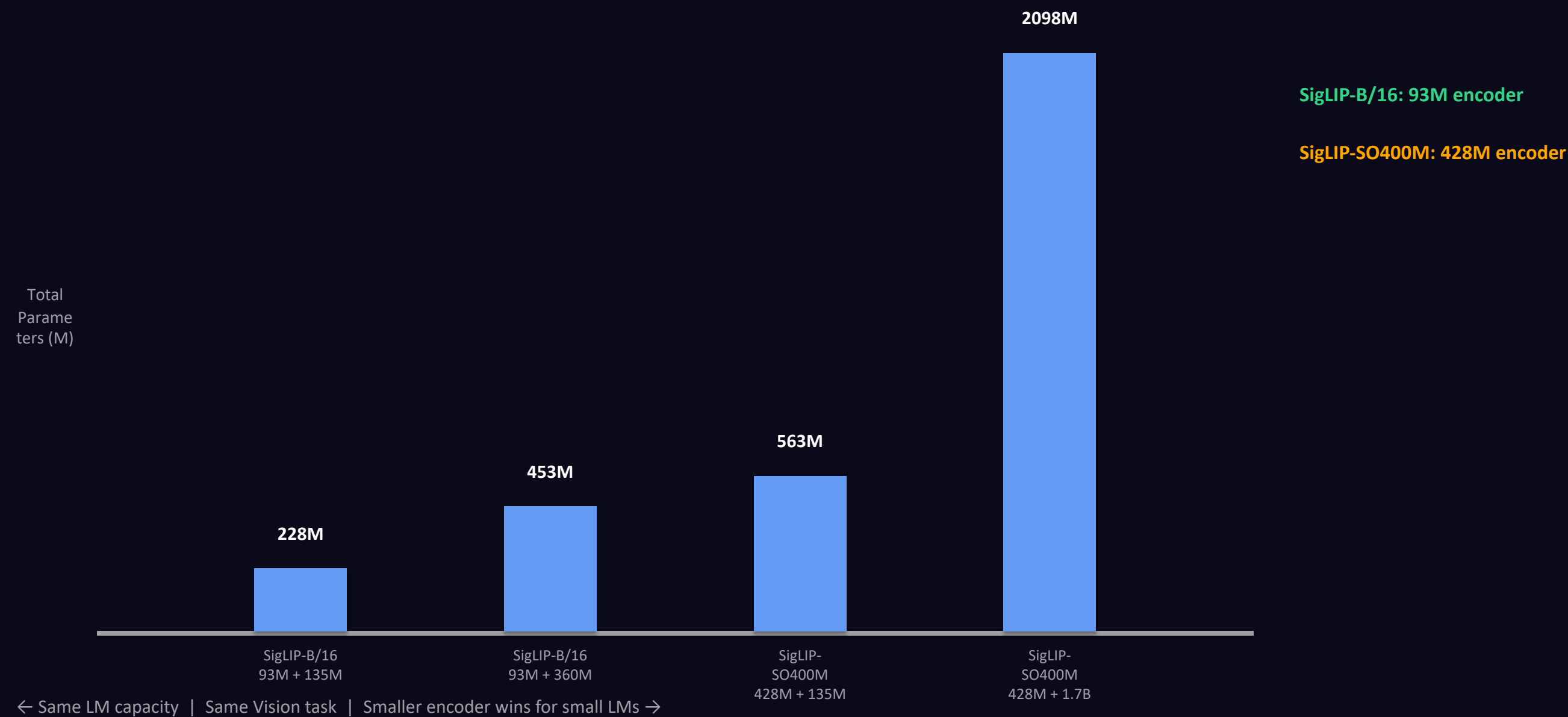
# SmolVLM Architecture: Image Splitting → Pixel Shuffle → SmolLM2 Backbone



Key insight: Pixel shuffle ( $r=4$ ) gives  $16\times$  token reduction for small models. Extended context (8K–16K tokens) via RoPE base increase (10K→273K) enables high-res images.

Architecture: Image → SigLIP Encoder → Pixel Shuffle → Linear → SmolLM2

# Smaller Vision Encoders Outperform Larger Ones in Compact VLMs

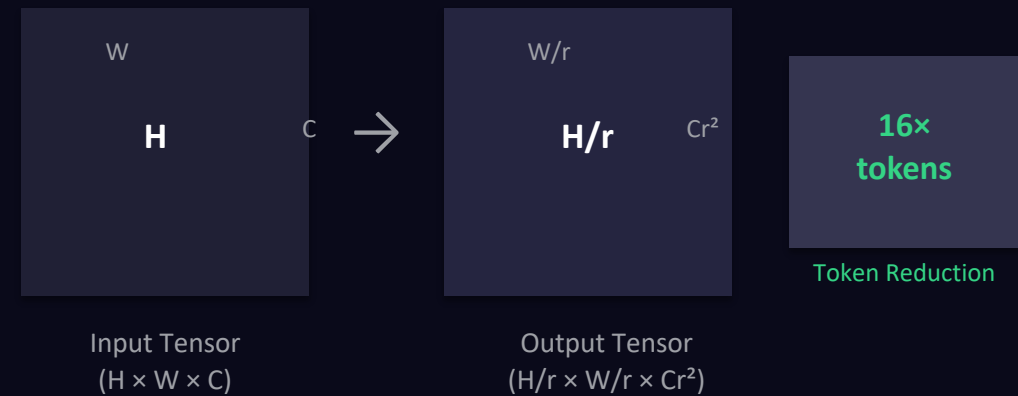


Finding 1: For compact VLMs (<500M total), the 93M SigLIP-B/16 encoder outperforms the 428M SigLIP-SO400M. Only at 1.7B LM scale does the larger encoder justify its ~10% parameter overhead.

# Aggressive Pixel Shuffle (r=4) Cuts Visual Tokens 16× for Small Models

## Pixel Shuffle: Space-to-Depth Rearrangement

Input ( $H \times W \times C$ ) is rearranged to ( $H/r \times W/r \times Cr^2$ )



| Config | Token Red. | Best for              | Model             |
|--------|------------|-----------------------|-------------------|
| r=1    | 1× (none)  | Large VLMs (>1B)      | —                 |
| r=2    | 4×         | Mid-size VLMs (~500M) | InternVL2         |
| r=4    | 16×        | Small VLMs (<500M)    | SmolVLM 256M/500M |
| r=8    | 64×        | Too aggressive        | —                 |

## Why r=4 for Small VLMs?

Finding 3: Small VLMs have limited attention budget.

- Attention complexity =  $O(n^2)$  where  $n$  = token count
- r=4 reduces visual tokens 16× → faster attention, lower memory
- Large VLMs: compute-per-token is the bottleneck → r=2 preserves detail
- Small VLMs: token count is the bottleneck → r=4 is optimal
- SmolVLM-256M: 0.8 GB RAM at batch=1 (vs. 2.7 GB without compression)

Empirical Validation: Pixel shuffle (r=4) outperforms no compression on all small-VLM benchmarks while reducing KV cache size 16×.

# Extending Context from 2K to 16K Tokens Unlocks Higher Image Resolution

## Context Length Scaling

*Finding 2: RoPE base 10K→273K + mixed-context fine-tuning*



*Higher context = higher resolution images = more visual detail retained.*

## Implementation Details

### RoPE Base Increase

10K → 273K  
(supports 16K context with 2× extrapolation headroom)

### Fine-tuning Strategy

Mixed long/short-context data  
(gradually increases context during training)

### Resolution Scaling

Higher context → higher input resolution  
(more sub-image patches at 384×384)

### Model Capacity

2.2B uses 16K context  
256M/500M use 8K context

| SmolVLM Achieves Competitive Performance at Sub-1GB to 4.9GB RAM |                      |                      |                      |                       |                      |
|------------------------------------------------------------------|----------------------|----------------------|----------------------|-----------------------|----------------------|
| Benchmark                                                        | Task                 | 256M                 | 500M                 | 2.2B                  | Best Efficient OS    |
| OCRBench                                                         | Single-Img           | 52.6%                | 61.0%                | 72.9%                 | 54.7% — Molmo-A1B-7B |
| AI2D                                                             | Single-Img           | 46.4%                | 59.2%                | 70.0%                 | 71.0% — Molmo-A1B-7B |
| ChartQA                                                          | Single-Img           | 55.6%                | 62.8%                | 68.7%                 | 48.0% — Molmo-A1B-7B |
| TextVQA                                                          | Single-Img           | 50.2%                | 60.2%                | 73.0%                 | 61.5% — Molmo-A1B-7B |
| DocVQA                                                           | Single-Img           | 58.3%                | 70.5%                | 80.0%                 | 77.7% — Molmo-A1B-7B |
| ScienceQA                                                        | Single-Img           | 73.8%                | 80.0%                | 89.6%                 | 87.5% — Molmo-A1B-7B |
| MMMU                                                             | Multi-task           | 29.0%                | 33.7%                | 42.0%                 | 33.9% — Molmo-A1B-7B |
| MathVista                                                        | Multi-task           | 35.9%                | 40.1%                | 51.5%                 | 37.6% — Molmo-A1B-7B |
| MMStar                                                           | Multi-task           | 34.6%                | 38.3%                | 46.0%                 | 43.1% — Molmo-A1B-7B |
| Video-MME                                                        | Video                | 33.7%                | 42.2%                | 52.1%                 | 45.0% — InternVL2-2B |
| MLVU                                                             | Video                | 40.6%                | 47.3%                | 55.2%                 | 48.2% — InternVL2-2B |
| Average (all 11)                                                 |                      | 44.0%                | 51.0%                | 59.8%                 |                      |
| RAM Usage:                                                       | SmolVLM-256M: 0.8 GB | SmolVLM-500M: 1.2 GB | SmolVLM-2.2B: 4.9 GB | Molmo-A1B-7B: 27.7 GB |                      |

# Real-Time Inference on iPhones: Up to 80 Tokens/Second on Apple Silicon

HuggingSnap — on-device VLM inference, zero cloud dependency

| MacBook Air M3<br>16GB | iPhone 15 Pro<br>8GB | MacBook Pro M3 Max<br>64GB | MacBook Pro M3 Max<br>64GB |
|------------------------|----------------------|----------------------------|----------------------------|
| ~55 tok/s              | ~40 tok/s            | ~80 tok/s                  | ~1000 tok/s                |
| 256M batch=1           | 256M batch=1         | 256M batch=1               | 256M batch=64              |

## Deployment Highlights

- Fully on-device: no cloud, no network latency
- Privacy-preserving: photos never leave the device
- SmolVLM-256M runs in ~100ms per token on iPhone 15 Pro
- Batch processing scales near-linearly for video ingestion
- HuggingSnap available on iOS App Store



# Key Takeaways: Design Principles for Efficient Multimodal Models

01

## Balance Encoder-LM Compute

For compact VLMs (<500M total), smaller encoders (93M SigLIP-B/16) outperform larger ones (428M). Only at 1.7B+ LM scale does the larger encoder justify parameter overhead.

02

## Extend Context Length

Increase RoPE base from 10K→273K to support 16K context. Fine-tune on mixed long/short data. Higher context unlocks higher resolution images without out-of-distribution failure.

03

## Use Aggressive Token Compression for Small Models

Pixel shuffle  $r=4$  (16× reduction) is optimal for VLMs <500M. Large models (2.2B) prefer  $r=2$ . Compression dramatically reduces attention  $O(n^2)$  cost and KV cache size.

04

## Sub-1GB Inference Is Achievable

SmoIVLM-256M achieves 0.8 GB RAM at batch=1, enabling smartphone deployment. It surpasses the 300× larger Llafix-80B on benchmarks, proving efficiency > raw scale.

05

## Open-Source Enables Rapid Iteration

All weights, datasets, training code, and the HuggingSnap mobile app are fully open-source. Open release lowers barrier to entry for on-device AI research and application development.

*Implication: Architectural efficiency — not raw scale — is the primary lever for practical on-device multimodal AI.*